Attachment 5 — Threat Model Summary

> NLnet NGI Zero Commons Fund — Symphact application, 13th open call (deadline 2026-06-01).
> Condensed from `docs/trust-model-en.md` v1.1. Full text: `trust-model-en.md`.

1. Scope of this document

This attachment summarises the **security model** of Symphact / CFPU: who issues capabilities, how an actor proves authorisation to send a message, and what the threat assumptions are. The full trust chain (manufacture → cert issuance → boot-time verification → runtime CST lookup) is described in `trust-model-en.md`.

2. Capability-based security in one paragraph

A `TActorRef` is **the** capability — possessing it is the authorisation to send a message to that actor. There is no global namespace (no `/dev/sda` analogue, no "root" user). The token is an opaque 32-bit CST (Capability Slot Table) index; the actual permissions, actor ID, and core coordinates live in the HW-managed CST table, not in the token. On every incoming message, the target-core mailbox-edge HW unit performs a CST lookup. On `.NET` hosts the same model is enforced by the runtime; on CFPU silicon it is enforced by hardware.

3. Identified threats and mitigations

#	Threat	Mitigation
T1	**Capability token forgery** — an attacker guesses or constructs a `TActorRef` to gain access to a target actor it should not reach.	The CST slot is allocated only by `TCapabilityRegistry` at `Spawn` time. The token alone is meaningless without an active CST entry; on every Send the target-core HW unit performs a CST lookup. Invalid CST slot → drop + fail-stop + AuthCode quarantine against the signer.
T2	**Supervisor escape** — a crashed actor manipulates supervisor state during restart to bypass the supervision tree.	The strategy call (`Restart` / `Stop` / `Escalate`) runs on the crashed child's thread (M0.4 design — no cross-thread mailbox injection). The "Capability = reference" invariant is preserved across restart; the actor cannot manufacture new `TActorRef` s during the crash window. Restart = state re-initialisation from `Init()` or replay from journal, never inheritance of pre-crash state.
T3	**Hot-load tampering** — an attacker injects malicious CIL bytecode via the hot code loader.	(Hot code loading is deferred to a follow-up grant; this is a forward-looking mitigation plan.) AuthCode requires every bytecode to be signed with a FenySoft cert (or delegated subordinate CA). Seal Core verifies: (a) `SHA-256(bytecode) == cert.PkHash` , (b) `BitIce.Verify(cert, eFuse.CaRootHash)` , (c) `cert.SubjectId ∉ revocation_list` . All three must pass for `Spawn` to proceed. Signature scheme: LMS (NIST SP 800-208) — stateful hash-based, quantum-resistant via SHA-256 collision resistance.
T4	**Cross-actor state leakage on shared-GC host**	Per-actor arena allocators (ArrayPool-backed) and explicit state serialisation across restart (M0.5 persistence) bound the leak surface. On CFPU silicon the threat is absent — every actor has its own private SRAM. On .NET hosts the threat is mitigated, not eliminated, and remains documented as a known limit of the .NET reference platform.
T5	**Message-passing flood / denial of service**	Mailbox depth limits + backpressure (M4.3): a full FIFO blocks the sender or returns `SendError` on `TrySend` . On CFPU the FIFO has a fixed hardware depth (8 on F3, ~64 on F6) — natural HW backpressure. The supervisor watchdog (M2.2) kills actors that don't yield within N ms.
T6	**Capability delegation abuse** — an actor delegates a capability with broader permissions than the receiver should have.	Attenuation on delegation (M5.2): permissions can only be narrowed when forwarding (`Write` → `Read-only`), never broadened. The 8-bit permission flag (`Send` , `Stop` , `Watch` , `Delegate` , `Revoke` , `Query` , `Snapshot` , `Migrate`) is masked during forwarding.
T7	**Revoked capability replay** — an actor caches an old `TActorRef` and continues to use it after revocation.	Revocation invalidates the CST slot; the HW lookup on the next Send fails (drop + fail-stop). On the .NET host, the registry maintains a generation counter; stale tokens fail the lookup. Revocation broadcast (M5.2) propagates to all known routers.
T8	**Side-channel attacks (cache timing, power analysis)**	**Out of scope for the base F6 chip.** The optional F6.5 Secure Edition variant adds physical tamper resistance, side-channel countermeasures, PUF, and TRNG. The Symphact software runtime is not the appropriate layer to address physical-attack threats.

#	Threat	Mitigation
T9	**Supply chain attacks on `.NET` dependencies**	Symphact runtime depends only on the .NET 10 BCL + xUnit (tests). The persistence layer is **BCL-only** by design — no external NuGet dependencies in `Symphact.Persistence`. The reduced dependency surface bounds the supply-chain attack surface. AuthCode signing pipeline (when implemented) requires SHA-256 binding to the exact bytecode.

4. NOT-supported options (explicit rejections)

The CFPU product line **explicitly rejects** several options that would create attack vectors, even if customers request them for convenience:

Rejected option	Why rejected
Multi-root eFuse array (more than one trust anchor in OTP)	Spare slots are an attack surface; a post-manufacture attacker could program their own root key
Open-mode eFuse bit ("developer mode" accepting self-signed bytecode)	Any runtime-configurable trust decision is an attack vector
Deployment-mode toggle (runtime override of trust model)	The AuthCode SHA-256 binding is a mandatory invariant; no mechanism may break it
"Open mode" FenySoft Verified chip (anyone runs any bytecode)	Brand integrity is a direct consequence of Strict whitelist mode

The Apache-2.0 / CERN-OHL-S licenses permit anyone to manufacture their own chip from CLI-CPU RTL with their own trust anchor — but that is **not "FenySoft Verified"**, it is a separate product category (the Android model: AOSP is open; "Google Play Services" + the trademark are closed).

5. Formal verification (M6 grant deliverable)

The grant funds **initial formal specification** of the `send` / `receive` semantics in TLA+ or Dafny, including:

- The capability invariant: a `TActorRef` only resolves to its issued target; revocation is monotonic.
- The supervision lifecycle invariant: `PreRestart` → state-reset → `PostRestart` is an atomic transition from the actor's external observers.
- The `TCapabilityRegistry` invariant: every allocated CST slot has a single owner; delegation produces a new slot, not a shared one.

This is **groundwork for a later independent security audit** when project maturity warrants it — for a v0.5-stage startup project an external audit would be premature. The grant builds the audit input substrate, not the audit itself.

6. Comparison with existing security models

System	Security model	Symphact difference
POSIX (Linux, macOS)	UID/GID, global namespace, ambient authority	No global namespace, capability = reference, no ambient authority
seL4	Formally verified capability microkernel in C	Same model in .NET / CIL; ECMA-335 type-safe at the ISA level on CFPU
CHERI / Morello	Hardware capability extension to ARM/RISC-V	Software-first today; capability at runtime + HW-enforced on CFPU
Akka.NET / Proto.Actor	Userland actor framework, shared GC, POSIX permissions	Capability-restricted by construction; runtime + HW-level Send validation
Anthropic Agent SDK / OpenAI Agents	Function-calling + JSON in Python, no capability isolation	Capability-restricted multi-agent runtime, structured audit chain

7. Threat-model maturity at submission

The threats above are **identified and design-mitigated**; the mitigations corresponding to **shipped milestones (M0.1–M0.4 + M0.5 BCL-only)** are implemented and test-covered (186 green xUnit tests). The remaining mitigations are scheduled within the grant scope:

- T1 (token forgery): mitigated by M2.5 Capability Registry (grant M2)
- T2 (supervisor escape): mitigated by M0.3 supervision (✅ shipped) + M2.5 (grant M2)
- T3 (hot-load tampering): forward-looking — AuthCode design recorded, implementation deferred to follow-up grant
- T4 (cross-actor leakage): mitigated by M0.5 persistence + per-actor arena (grant M1)
- T5 (DoS / flood): mitigated by M4.3 backpressure (post-grant) + HW FIFO depth (CFPU)
- T6 (delegation abuse): mitigated by M5.2 attenuation (grant M2 partial, post-grant for full)
- T7 (revoked replay): mitigated by M5.2 revocation broadcast (grant M2)
- T8 (side-channel): out of base scope; addressed by optional F6.5 Secure Edition (CLI-CPU project)
- T9 (supply chain): mitigated by BCL-only persistence (✅ shipped) + AuthCode (post-grant)

Related documents:

- `docs/trust-model-en.md` — full trust chain and business-model context (v1.1)
- `docs/vision-en.md` — capability-based security design foundations
- `docs/actor-ref-scaling-en.md` — TActorRef bit layout and defence pyramid
- `CLI-CPU/docs/authcode-hu.md` — AuthCode trust chain specification (in the hardware repo)
- `CLI-CPU/docs/secure-element-hu.md` — F6.5 Secure Edition (optional hardening)